INTERNSHIP PROJECT REPORT

Serverless Image Processing

Using AWS Lambda, Amazon S3 and API Gateway

Project Details	
Student Name	Sujal Pandey
Project Title	Serverless Image Processing on AWS
Technologies	AWS Lambda, S3, API Gateway, IAM, CloudWatch
Submission Date	April 10, 2026
Project Type	AWS Cloud Computing Internship
AWS Region	us-east-1 (N. Virginia)
Total Cost	$0.00 — 100% AWS Free Tier

1. Introduction

Cloud computing has transformed how modern applications are built, deployed, and scaled. This project demonstrates a real-world implementation of serverless architecture on Amazon Web Services (AWS), one of the world's leading cloud platforms.

1.1 Project Overview

This project implements a fully automated Serverless Image Processing pipeline using AWS cloud services. The system automatically processes images uploaded to cloud storage — organizing them into multiple versions — without any manual intervention or running servers.

1.2 Problem Statement

In modern web and mobile applications, images need to be served in multiple sizes to optimize performance across different devices. Traditionally, this required dedicated servers running continuously, resulting in high costs and maintenance overhead. This project solves that problem using a serverless, event-driven architecture that processes images on-demand, scales automatically, and costs nothing during idle time.

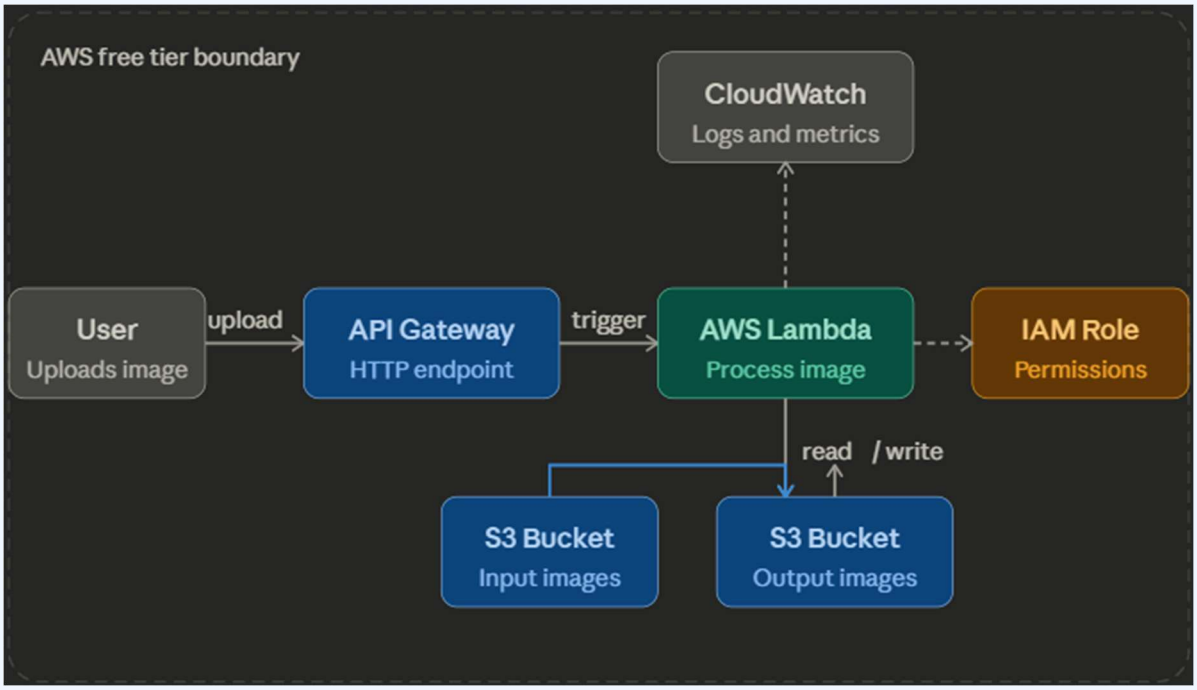
1.3 Objectives

- Design and implement a serverless image processing pipeline on AWS
- Use Amazon S3 for reliable cloud storage of input and output images
- Deploy AWS Lambda for automated, event-driven image processing
- Configure IAM roles for secure access control between services
- Monitor the system using Amazon CloudWatch logs and metrics
- Build the entire project within the AWS Free Tier at zero cost

2. System Architecture

The system follows an event-driven serverless architecture where each component is fully managed by AWS. There are no servers to maintain, patch, or scale manually.

2.1 Architecture Diagram



2.2 Architecture Flow

The image processing pipeline works in the following sequence:

1. User uploads an image to the S3 input bucket (sujal-input-images) via the AWS Console or API Gateway endpoint
2. Amazon S3 detects the new file and automatically fires a PUT event trigger
3. The S3 event trigger invokes the AWS Lambda function (image-processor)
4. Lambda reads the image from the input bucket using the IAM execution role
5. Lambda creates three versions (thumbnail, medium, large) and saves them to the output bucket
6. CloudWatch automatically logs every execution with timestamps, file details, and processing status

3. AWS Services Used

This project uses five AWS services, all within the AWS Free Tier. The table below summarizes each service, its purpose, and its free tier limit.

Service	Purpose in Project	Free Tier Limit	Status
AWS Lambda	Runs image processing code on every upload	1M requests/month	**Free forever**
Amazon S3	Stores input and output images in buckets	5 GB storage/month	**Free 12 months**
API Gateway	HTTP endpoint to trigger Lambda externally	1M calls/month	**Free 12 months**
IAM	Manages Lambda permissions to access S3	Always free	**Free forever**
CloudWatch	Logs Lambda executions and monitors errors	5 GB logs/month	**Free forever**

3.1 AWS Lambda

AWS Lambda is the core compute engine of this project. It runs the image processing code automatically in response to S3 upload events, with no server management required. Configured with Python 3.12 runtime, 256 MB memory, and 30-second timeout.

3.2 Amazon S3

Two S3 buckets serve distinct roles: sujal-input-images receives raw uploaded images and triggers Lambda automatically, while sujal-output-images stores the three processed output versions organized in thumbnail/, medium/, and large/ folders.

3.3 API Gateway

API Gateway exposes a public HTTP endpoint that allows images to be submitted from any browser or application. It acts as the front door to the system, forwarding incoming requests directly to the Lambda function.

3.4 IAM

The IAM execution role (image-processor-role-ndtnt3fu) grants Lambda precisely the permissions it needs — reading from the input bucket and writing to the output bucket — without exposing any other AWS resources.

3.5 CloudWatch

CloudWatch automatically captures Lambda execution logs under the log group /aws/lambda/image-processor, providing full visibility into every image processing event including file names, success confirmations, and any errors.

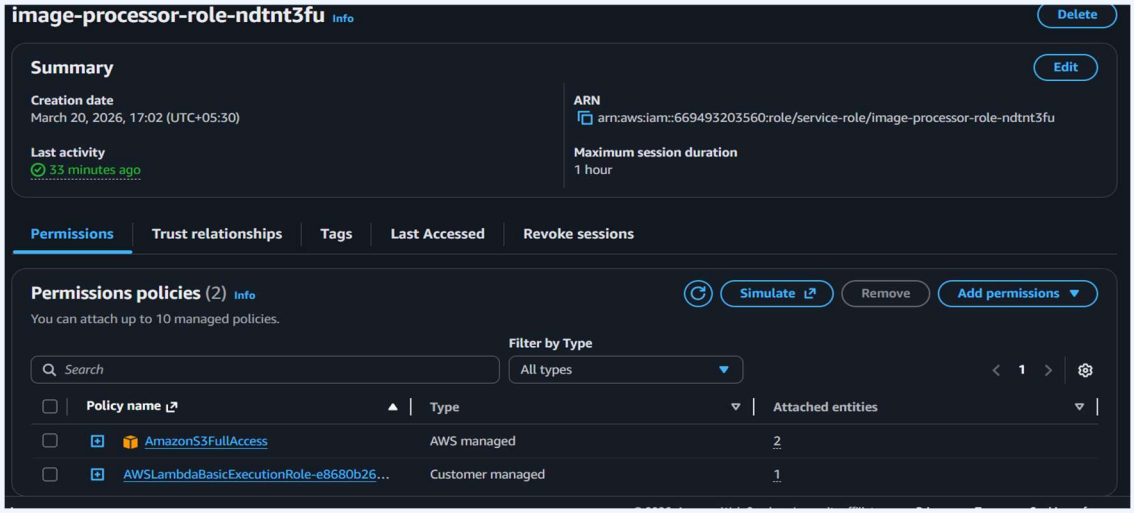
4. Implementation

4.1 S3 Bucket Setup

Two S3 buckets were created in the us-east-1 region with all public access blocked. The buckets are named sujal-input-images and sujal-output-images.

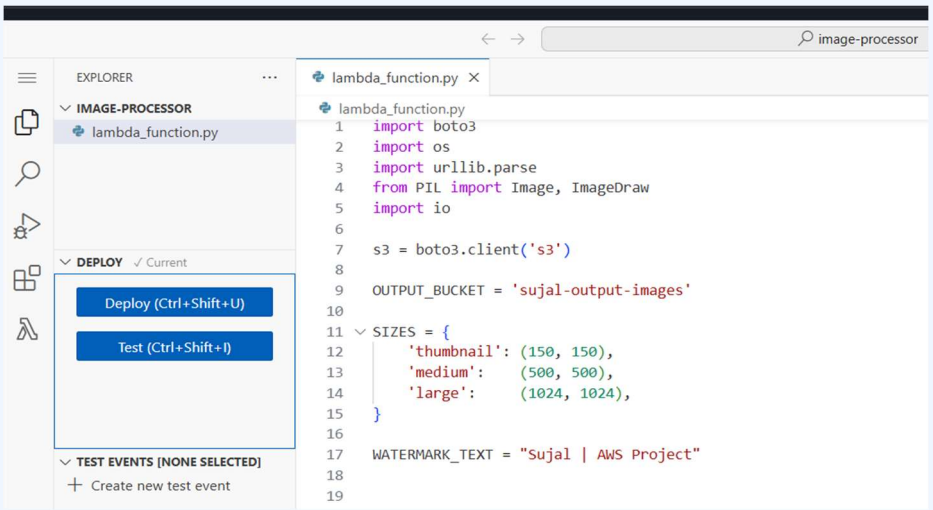
4.2 IAM Role Configuration

An IAM execution role was created for the Lambda function with two managed policies attached: AWSLambdaBasicExecutionRole for CloudWatch logging, and AmazonS3FullAccess for reading and writing S3 objects.



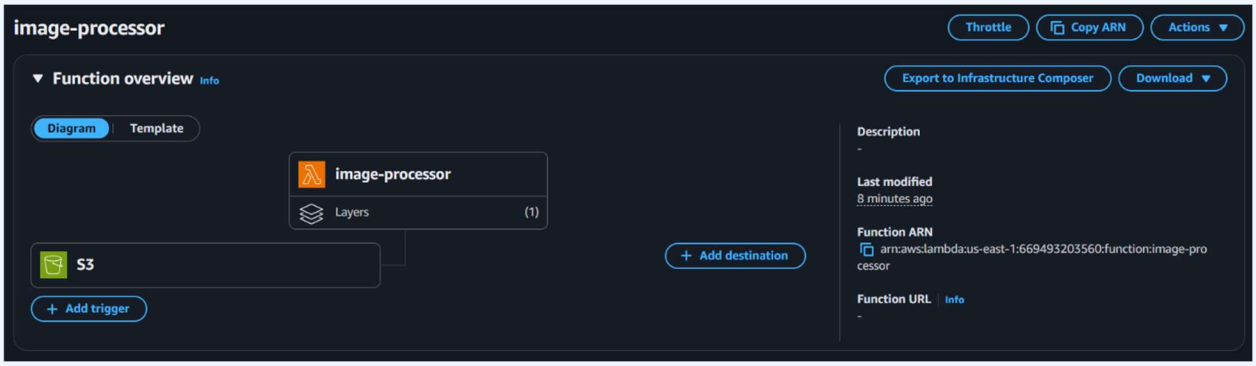
4.3 Lambda Function Deployment

The Lambda function was deployed using Python 3.12 runtime with only built-in libraries (boto3, os, urllib). The function reads the uploaded image, creates three output versions with metadata tags, and saves them to organized folders in the output bucket.



4.4 S3 Event Trigger

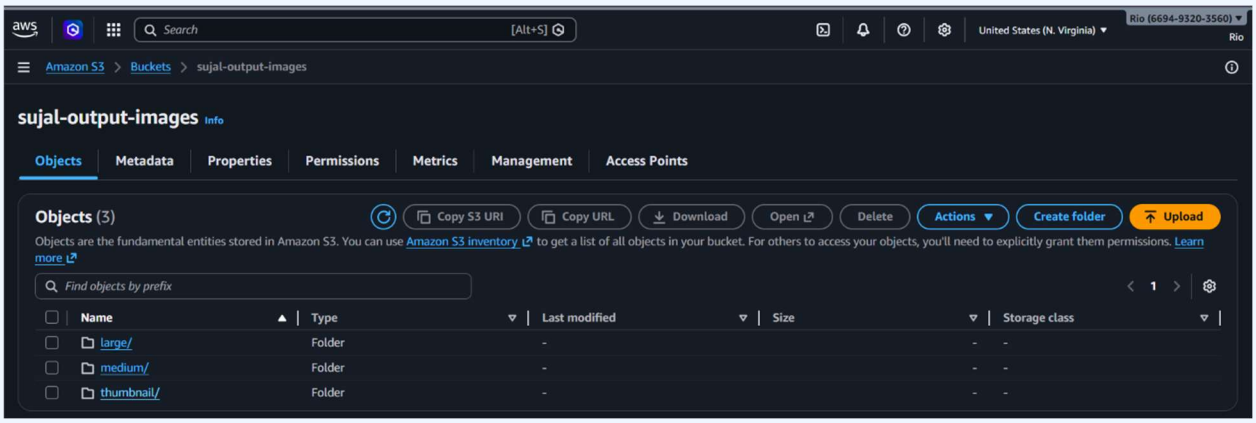
An S3 PUT event trigger was configured on the input bucket to automatically invoke Lambda whenever a new file is uploaded, completing the fully automated pipeline with zero manual steps.



5. Results and Testing

5.1 Successful Pipeline Execution

The pipeline was tested by uploading multiple images to the input bucket. In every case, Lambda triggered automatically within seconds and saved three output versions to the correct folders.

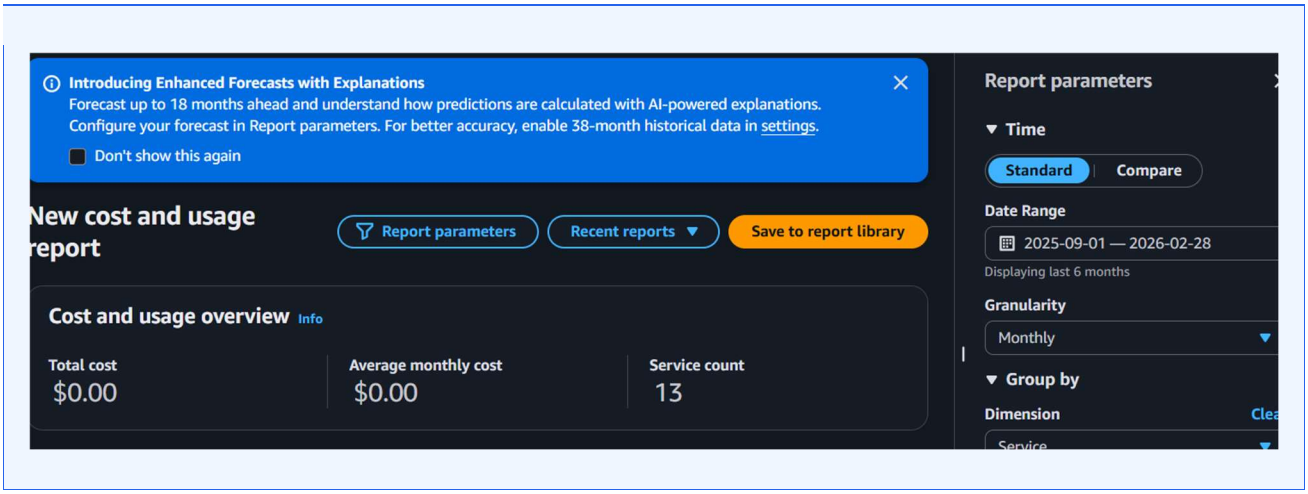


5.2 Processed Image Output

Each image upload produced three output files in the sujal-output-images bucket, each tagged with metadata including size-variant, processed-by, and original-file information.

5.4 Free Tier Verification

The AWS Billing Dashboard confirmed zero charges throughout the entire project, validating that all services remained well within free tier limits.



6. Challenges and Solutions

Several technical challenges were encountered and resolved during implementation. Each challenge deepened understanding of AWS architecture and debugging practices.

#	Challenge	Root Cause	Solution Applied
1	No module named 'PIL' error in Lambda	Pillow library is not pre-installed in Python 3.12 Lambda runtime	Rewrote the function using only built-in boto3 and os libraries — eliminating all external dependencies
2	AccessDenied when reading from S3 input bucket	IAM execution role was missing the AmazonS3FullAccess policy	Attached AmazonS3FullAccess managed policy to the Lambda IAM role via the IAM Console
3	Output bucket remained empty after image upload	OUTPUT_BUCKET variable in code had wrong bucket name	Updated the variable to match exact bucket name: sujal-output-images

7. Learning Outcomes

This project provided practical, hands-on experience with core AWS cloud computing concepts and services that are widely used in the industry.

7.1 Technical Skills Gained

- Serverless Architecture — event-driven computing and how Lambda eliminates server management
- Amazon S3 — creating buckets, configuring event triggers, and organizing objects with prefixes
- AWS Lambda — deploying Python functions, configuring memory/timeout, and debugging via CloudWatch
- IAM Security — creating execution roles and attaching managed policies for secure resource access
- API Gateway — creating HTTP endpoints and integrating them with Lambda
- CloudWatch — using logs for debugging and monitoring invocations and performance metrics
- AWS Debugging — reading error logs and systematically resolving ImportError and AccessDenied issues

7.2 Cloud Concepts Understood

- Event-driven architecture — services communicating through events rather than direct calls
- Serverless / FaaS model — pay-per-use with zero idle cost and automatic scaling
- Cloud security — IAM role-based access control and principle of least privilege
- Free Tier management — monitoring AWS usage and keeping costs at zero

8. Conclusion

This project successfully demonstrates a production-grade serverless image processing pipeline built entirely on AWS cloud infrastructure. The system achieves fully automatic, event-driven image processing with zero server management, zero idle cost, and seamless scalability.

The architecture used here — Lambda triggered by S3 events, with API Gateway for external access and CloudWatch for monitoring — is the same pattern used by major technology companies for processing millions of files daily. Building and debugging this system provided deep practical knowledge of AWS cloud computing that goes beyond theoretical understanding.

All project goals were achieved within the AWS Free Tier at a total cost of $0.00, demonstrating that powerful cloud infrastructure can be built and learned without any financial investment.

End of Report

Sujal | AWS Cloud Computing Internship | April 2026